

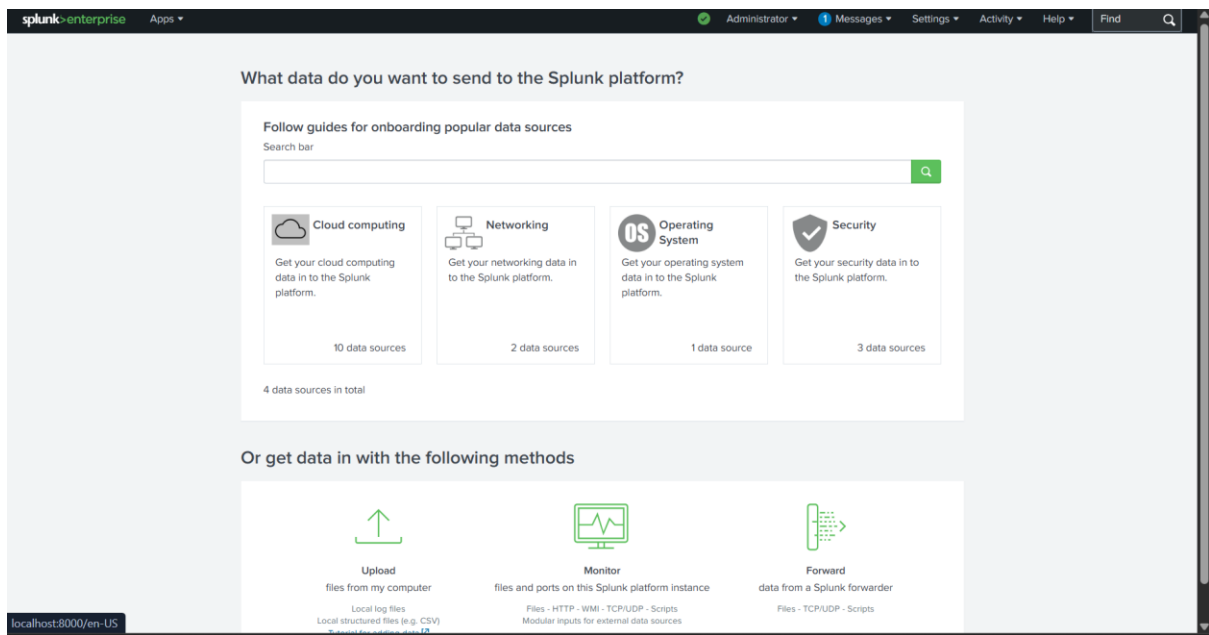
Lab 2 - Splunk Dashboard for Web Traffic Logs

Objective

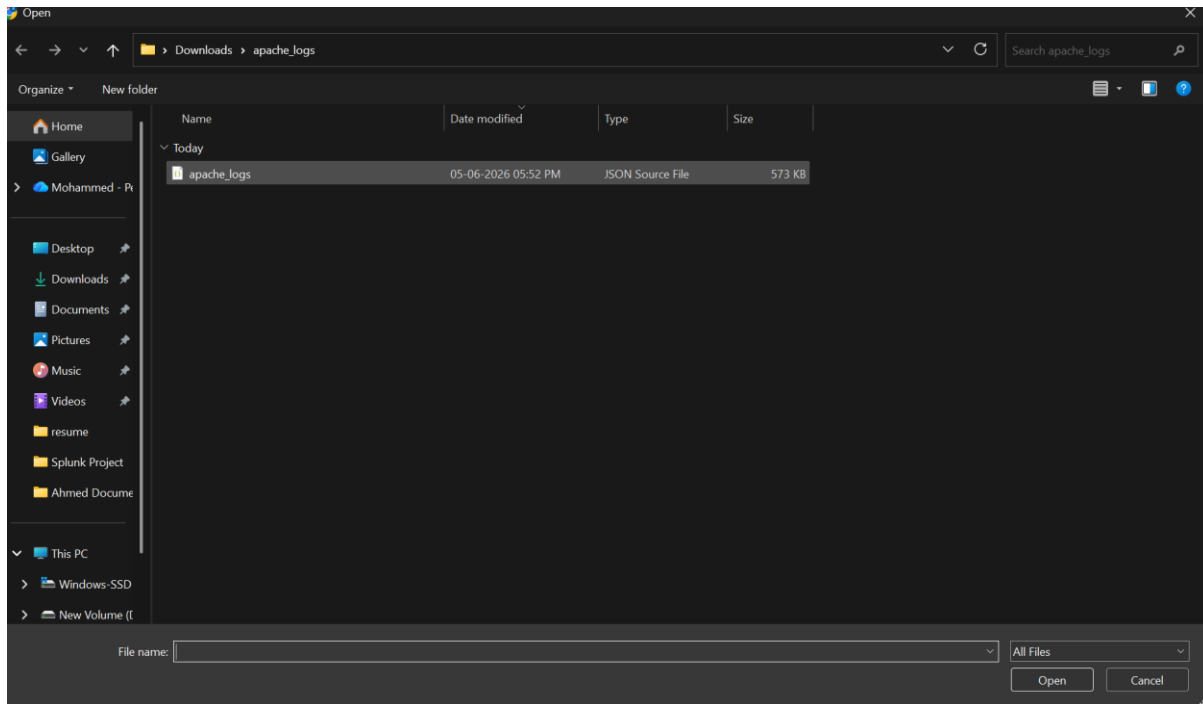
To create a Splunk dashboard that visualizes web server activity, response statistics, user access patterns, and geographic traffic distribution for effective monitoring and analysis of Apache web logs.

At first, Log in to the Splunk Web interface.

Navigate to Settings → Add Data or click Add Data from the Splunk home page.

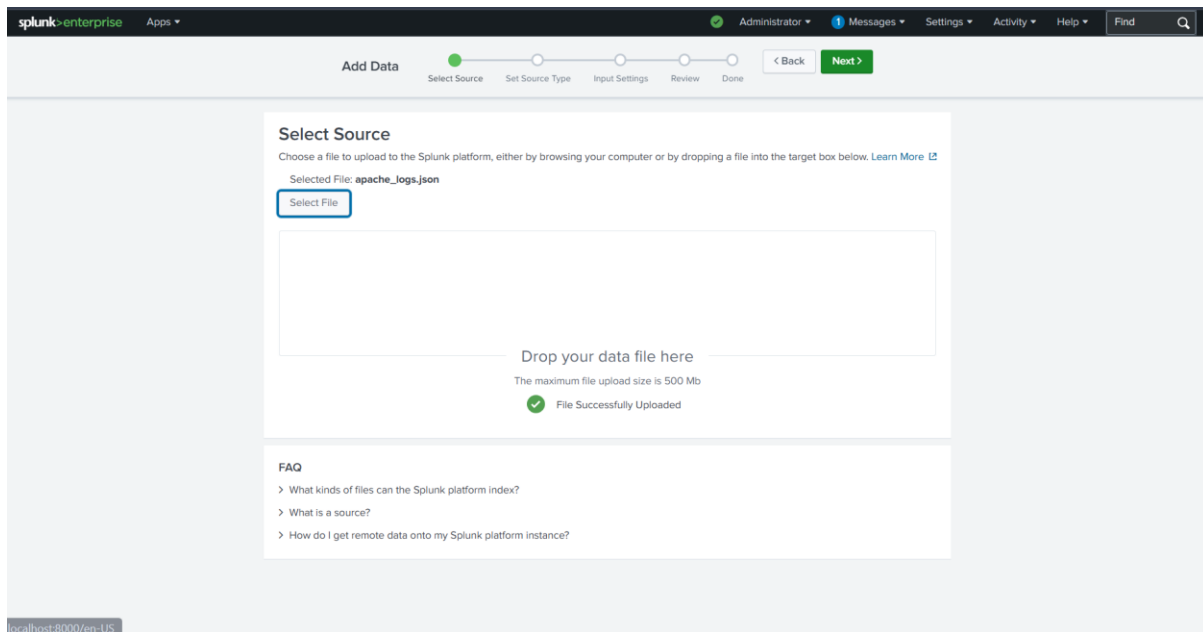


Click on >> Upload



Select the logs file [Link](#)

The file is uploaded into Splunk



Click on >> Next

This are the logs

The screenshot shows the 'Set Source Type' configuration page in Splunk Enterprise. The source is 'apache_logs.json'. The interface includes a 'View Event Summary' table with 10 columns: _time, bytes, ip, method, protocol, referrer, status, timestamp, uri, and user. The table displays 9 log entries. The 'Next' button is highlighted in the top navigation bar.

	_time	bytes	ip	method	protocol	referrer	status	timestamp	uri	user
1	9/17/25 12:00:00.000 PM	1374	185.62.57.52	GET	HTTP/1.1	-	200	17/Sep/2025:12:00:00 +0530	/upload.php	pytt
2	9/17/25 12:00:02.000 PM	5195	103.21.244.54	GET	HTTP/1.1	-	200	17/Sep/2025:12:00:02 +0530	/cart/view	Mo;
3	9/17/25 12:00:04.000 PM	1872	66.249.66.41	GET	HTTP/1.1	-	200	17/Sep/2025:12:00:04 +0530	/cart/view	Mo; Bin +htt
4	9/17/25 12:00:06.000 PM	870	185.62.57.82	GET	HTTP/1.1	http://malicious-spam-site.com	200	17/Sep/2025:12:00:06 +0530	/?	Mo; Win
5	9/17/25 12:00:08.000 PM	524	103.21.244.93	GET	HTTP/1.1	-	302	17/Sep/2025:12:00:08 +0530	/contact.html	Mo; Win
6	9/17/25 12:00:10.000 PM	2094	185.62.57.84	GET	HTTP/1.1	http://malicious-spam-site.com	200	17/Sep/2025:12:00:10 +0530	/?	Mo;
7	9/17/25 12:00:12.000 PM	4760	66.249.66.8	GET	HTTP/1.1	-	200	17/Sep/2025:12:00:12 +0530	/static/logo.png	Ahr +htt
8	9/17/25 12:00:14.000 PM	3991	66.249.66.5	GET	HTTP/1.1	-	200	17/Sep/2025:12:00:14 +0530	/cart/view	Mo; Go; +htt
9	9/17/25	4290	185.62.57.36	GET	HTTP/1.1	-	200	17/Sep/2025:12:00:16	/products.php?id=10	pytt

Click on >> Next

Let us keep our host name as webserver

The screenshot shows the 'Input Settings' configuration page in Splunk Enterprise. The 'Host' field is set to 'webserver' with the 'Constant value' radio button selected. The 'Index' is set to 'Default'. The 'Review' button is highlighted in the top navigation bar.

Host

When the Splunk platform indexes data, each event receives a "host" value. The host value should be the name of the machine from which the event originates. The type of input you choose determines the available configuration options. [Learn More](#)

Host field value:

Index

The Splunk platform stores incoming data as events in the selected index. Consider using a "sandbox" index as a destination if you have problems determining a source type for your data. A sandbox index lets you troubleshoot your configuration without impacting production indexes. You can always change this setting later. [Learn More](#)

Index: [Create a new index](#)

Click on >> Review

This is an overview of our uploaded file

The screenshot shows the 'Add Data' wizard in Splunk Enterprise. The top navigation bar includes the Splunk logo, 'Apps', and user roles like 'Administrator'. The wizard progress bar shows five steps: 'Select Source', 'Set Source Type', 'Input Settings', 'Review', and 'Done'. The 'Review' step is currently active. Below the progress bar, a 'Review' box displays the following configuration details:

Input Type	Uploaded File
File Name	apache_logs.json
Source Type	_json
Host	webserver
Index	Default

At the bottom right of the wizard, there are '< Back' and 'Submit >' buttons.

Click on >> Submit

Now that our logs are ready let us start Searching

The screenshot shows the 'Add Data' wizard in Splunk Enterprise, now at the 'Done' step. The progress bar shows the 'Review' step as completed. A large green checkmark and the message 'File has been uploaded successfully.' are displayed. Below this, a link suggests configuring inputs by going to 'Settings > Data Inputs'. A list of action buttons is provided:

- Start Searching**: Search your data now or see [examples and tutorials](#).
- Extract Fields**: Create search-time field extractions. [Learn more about fields](#).
- Add More Data**: Add more data inputs now or see [examples and tutorials](#).
- Download Apps**: Apps help you do more with your data. [Learn more](#).
- Build Dashboards**: Visualize your searches. [Learn more](#).

At the bottom of the screen, a URL bar shows the search path: `localhost:8000/en-US/app/search/search?q=source%3D"apache_logs.json" host%3D"webserver" sourcetype%3D"_json"&earliest=0&latest=`

Click on >> Start Searching

This is the search menu where we can get our desired output by querying

The screenshot shows the Splunk Search interface. The search bar contains the query: `source="apache_logs.json" host="webserver" sourcetype="_json"`. The results show 2,000 events. The left sidebar lists various navigation options like Search & Reporting, Analytics, Datasets, Reports, Alerts, Dashboards, and Modules. The main panel displays a timeline visualization and a list of events with their details, including IP addresses, timestamps, and methods.

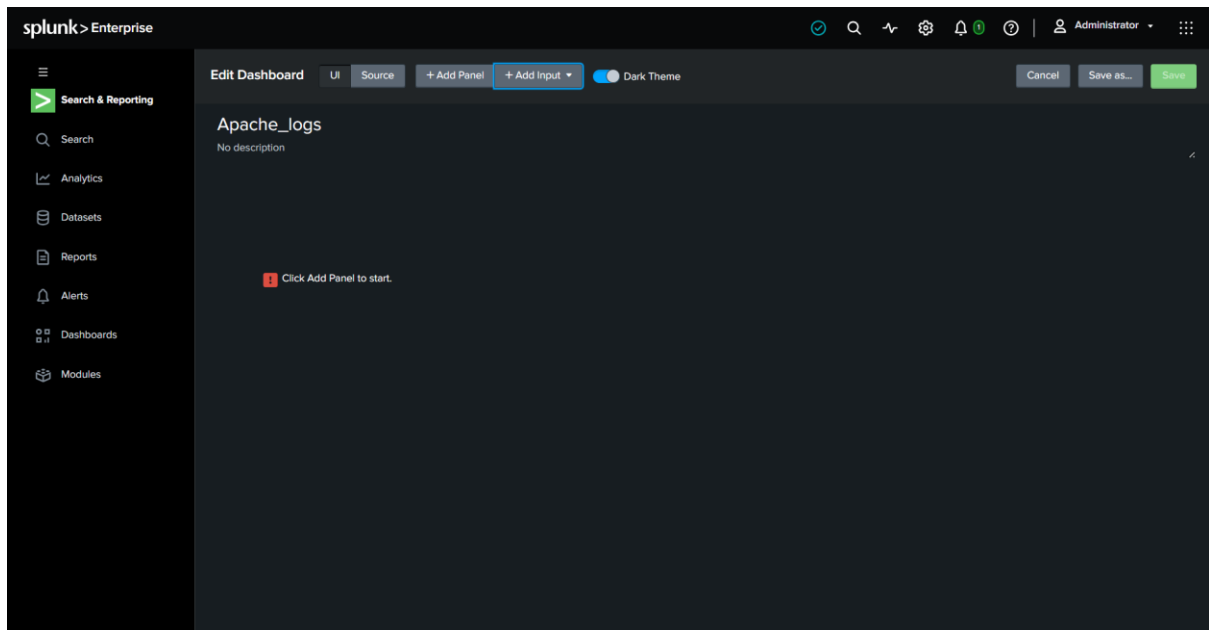
Click on >> Dashboard

Set a Dashboard tile and select the dashboard type as Classic

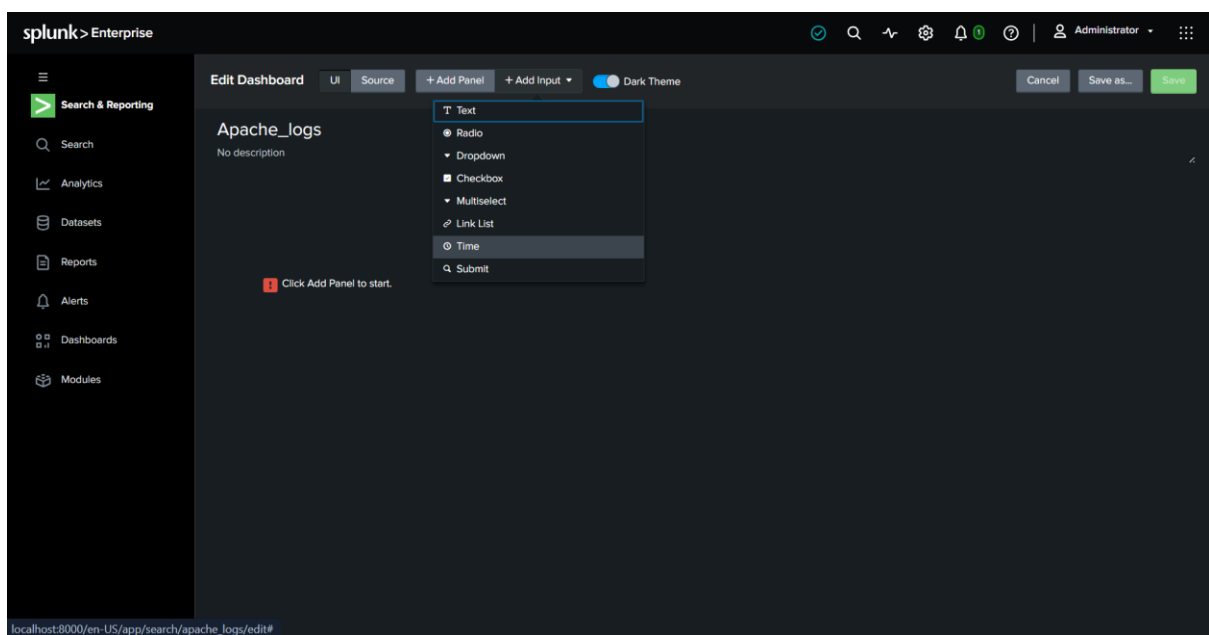
The screenshot shows the Splunk Dashboards interface. A 'Create New Dashboard' dialog is open, allowing the user to set the dashboard title to 'Apache_logs', description, permissions, and dashboard type. The 'Dashboard type' section shows two options: 'Classic Dashboards' (selected) and 'Dashboard Studio'. The background shows a list of existing dashboards with columns for App, Sharing, and Type.

Click on >> Create

Now is this blank dashboard, let us add time and submit button here

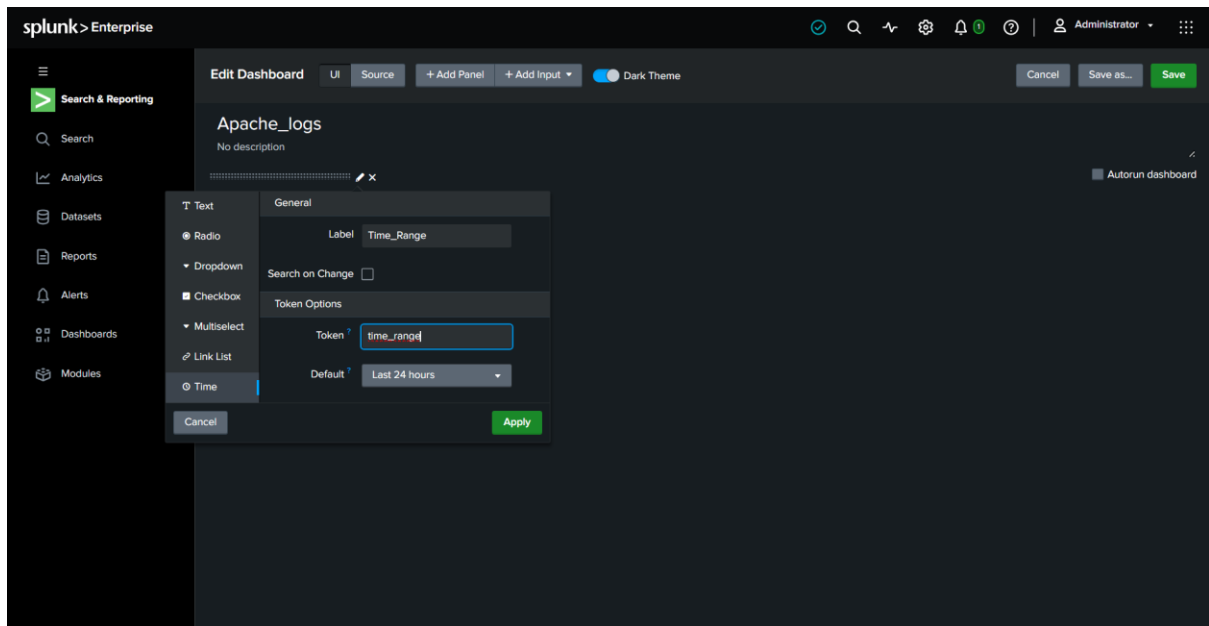


Click on >> Add Input



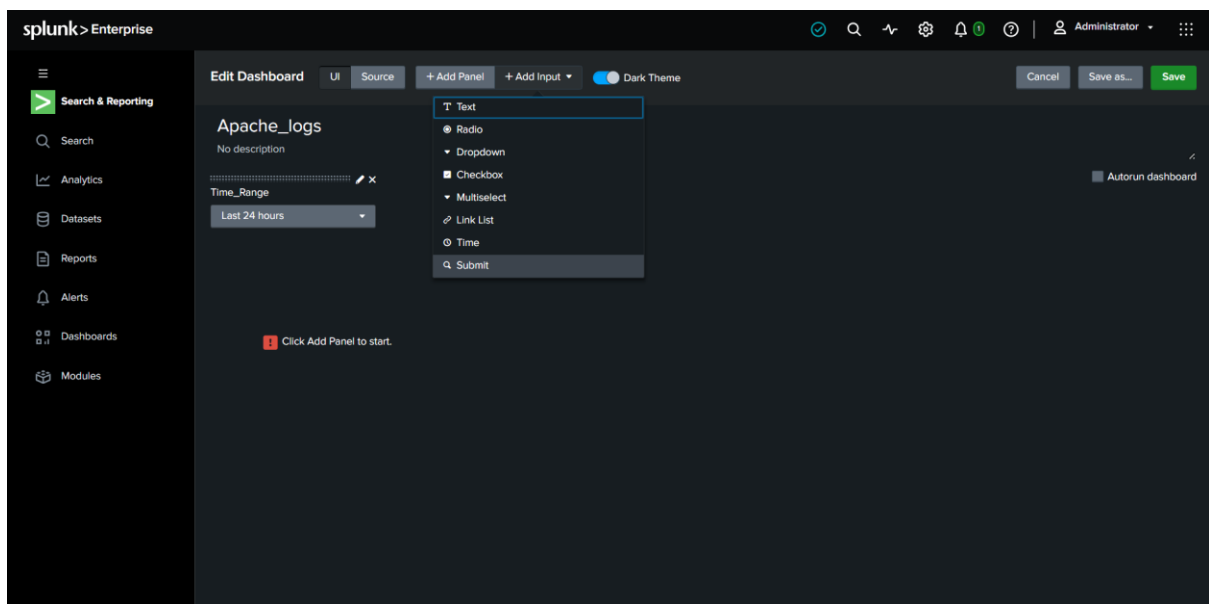
Click on >> Add Input >> Time

Click on pencil icon and set time range



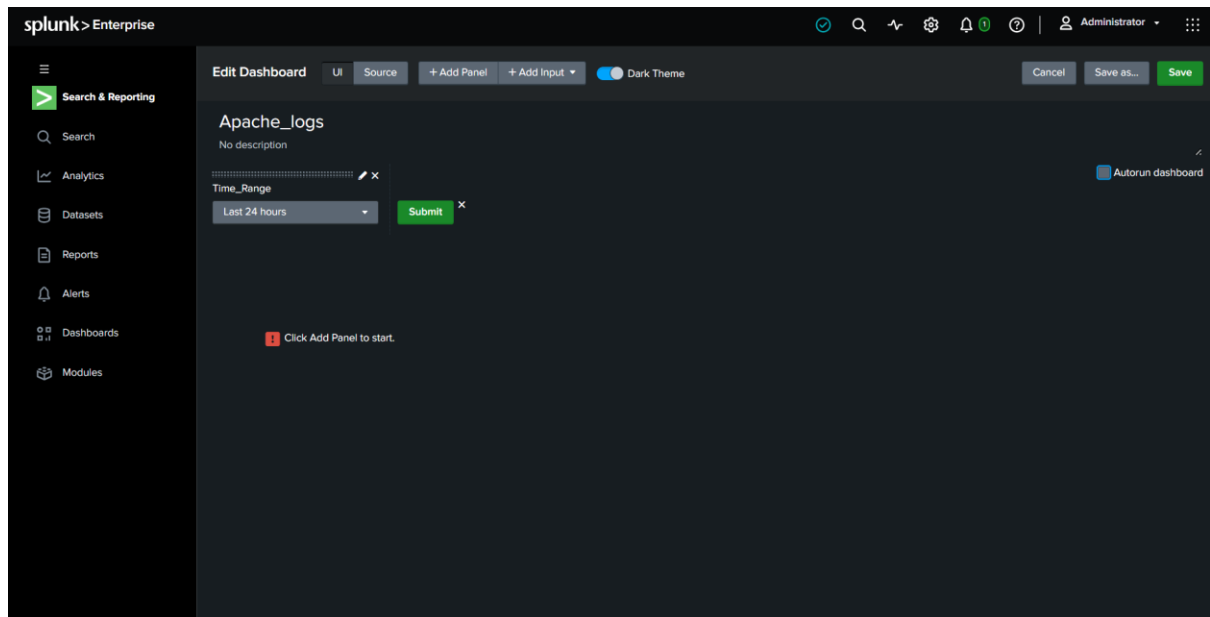
Click on >> Apply

As we can see time range is added in the dashboard, now let us add submit button



Click on >> Add Input >> Submit

Now let us do some querying



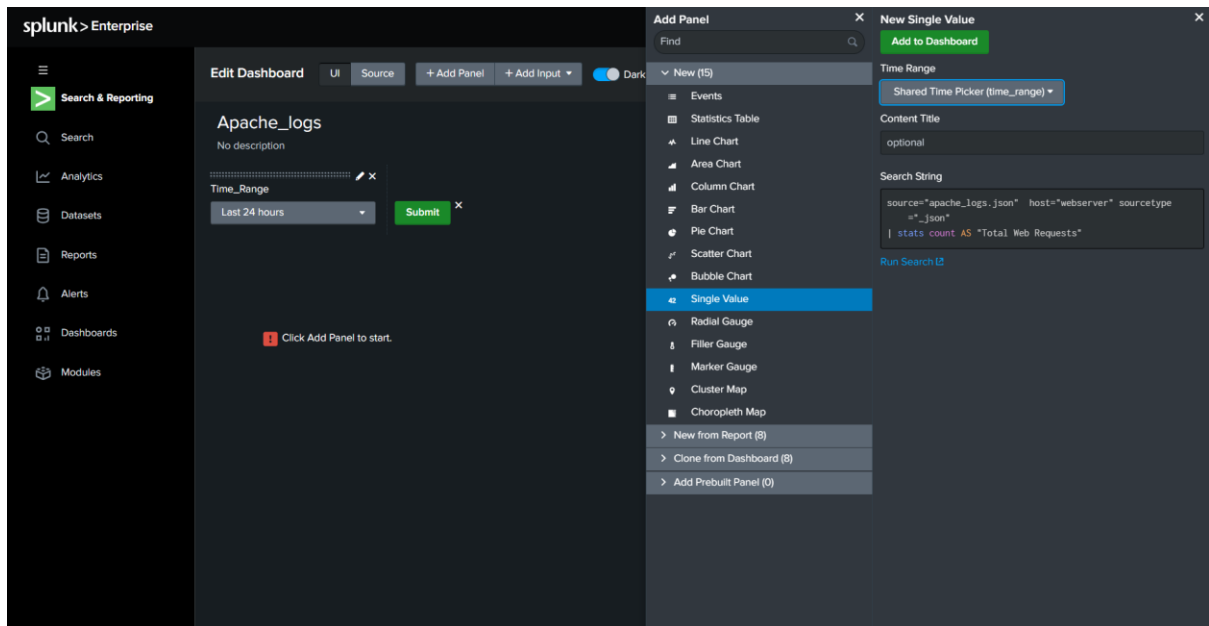
1. Total Web Requests

- Click on Add Panel
- Under New, choose Single Value
- Use Shared Time Picker **time_range**
- Set Content Title to "Total Web Requests"
- Enter the Search String as below

```
source="apache_logs.json" host="webserver" sourcetype="_json"
| stats count AS "Total Web Requests"
```

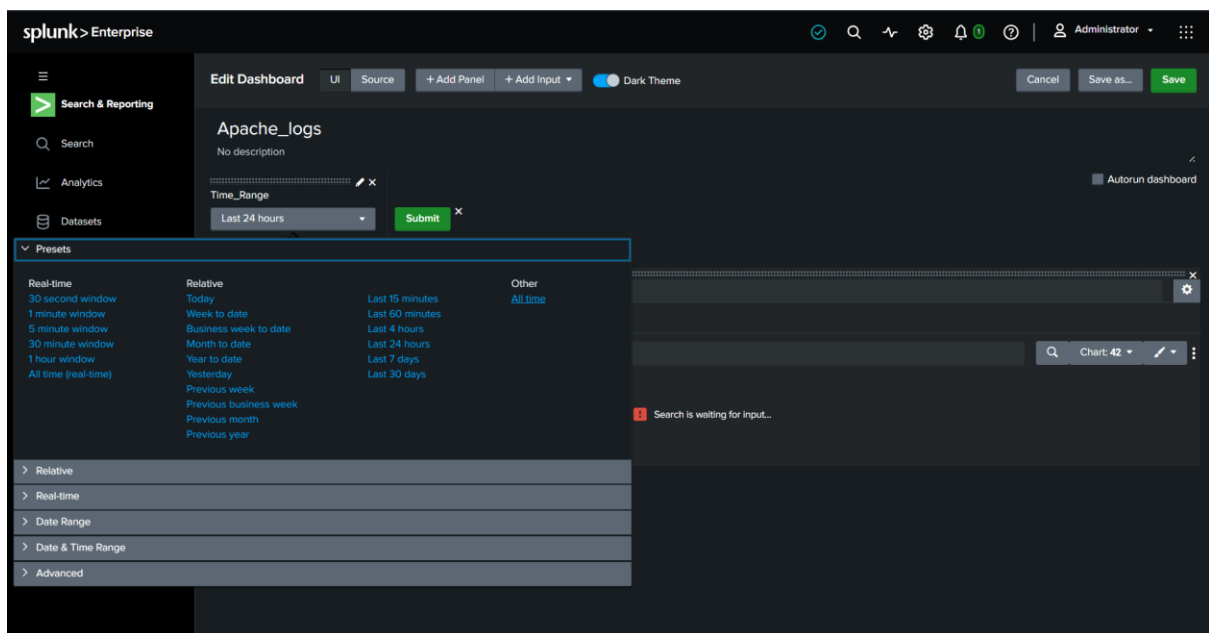
Explanation:

- Filters events from the apache_logs.json log source.
- Limits the search to logs generated by the webserver host.
- Uses the _json sourcetype to process JSON-formatted log data.
- Counts the total number of events (web requests) in the selected time range using the stats command.
- Renames the output field to **"Total Web Requests"** for better dashboard readability.



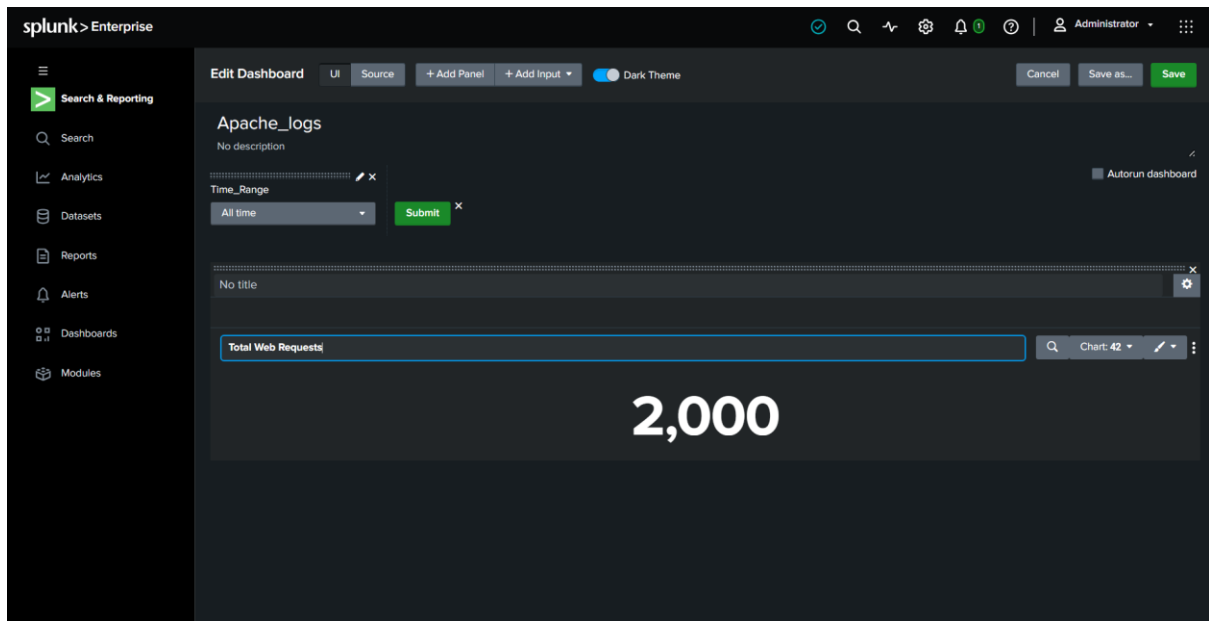
Click on >> Add to Dashboard

As we can see no logs are generated, so we need to set the time range to “All time”



Click on >> Time_Range >> All Time

We can see the logs now!



2. Successful Response

- Click on Add Panel
- Under New, choose Single Value
- Use Shared Time Picker **time_range**
- Set Content Title to "Successful Response"
- Enter the Search String as below

```
source="apache_logs.json" host="webserver" sourcetype="_json"
method=GET status=200
| stats count AS "Successful Responses"
```

Explanation:

- Filters events from the apache_logs.json log source.
- Limits the search to logs generated by the webserver host.
- Uses the _json sourcetype to process JSON-formatted log data.
- Selects only **GET** requests.
- Filters for HTTP **status code 200**, indicating successful requests.
- Counts the total number of successful responses using the stats command.
- Renames the output field to "**Successful Responses**" for clear dashboard visualization.

- Enter the Search String as below:

```
source="apache_logs.json" host="webserver" sourcetype="_json"
| where status>=400 and status<500
| stats count AS "Client Errors"
```

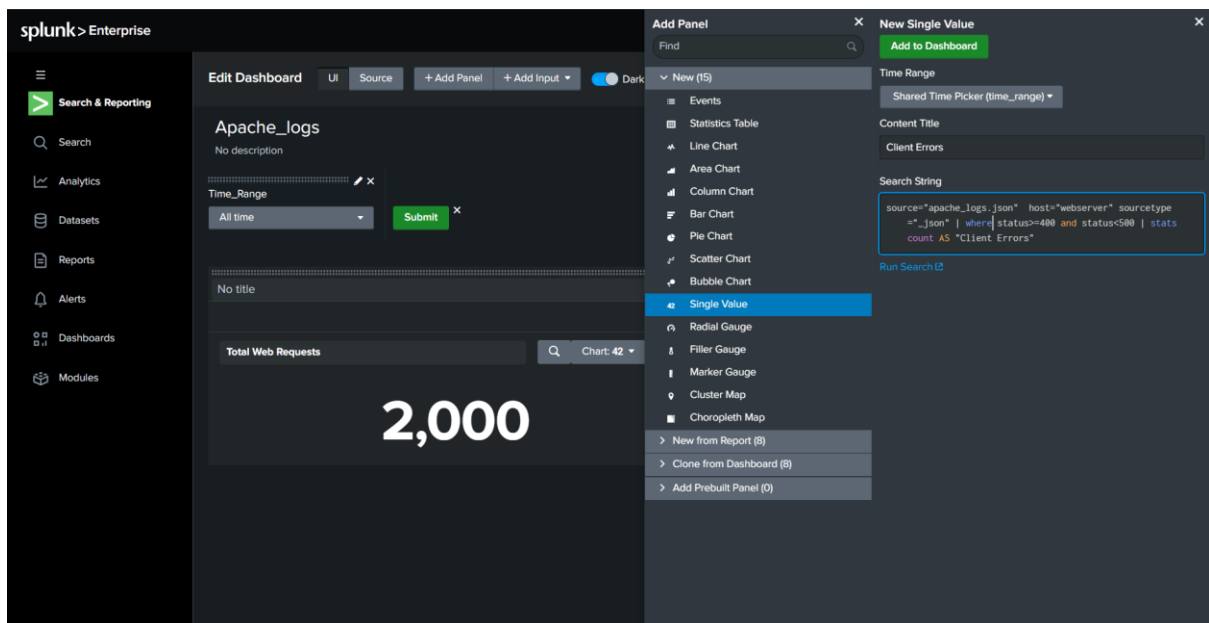
Explanation:

- Filters events from the apache_logs.json log source.
- Limits the search to logs generated by the webserver host.
- Uses the _json sourcetype to process JSON-formatted log data.
- The where command filters events with HTTP status codes between **400 and 499**.
- These status codes represent **client-side errors**, such as:
 - **400** – Bad Request
 - **401** – Unauthorized
 - **403** – Forbidden
 - **404** – Not Found
- The stats count command counts the total number of matching error events.
- Renames the output field to **"Client Errors"** for better readability in the dashboard.

```
source="apache_logs.json" host="webserver" sourcetype="_json"
| where status<500
| stats count AS "Server Errors"
```

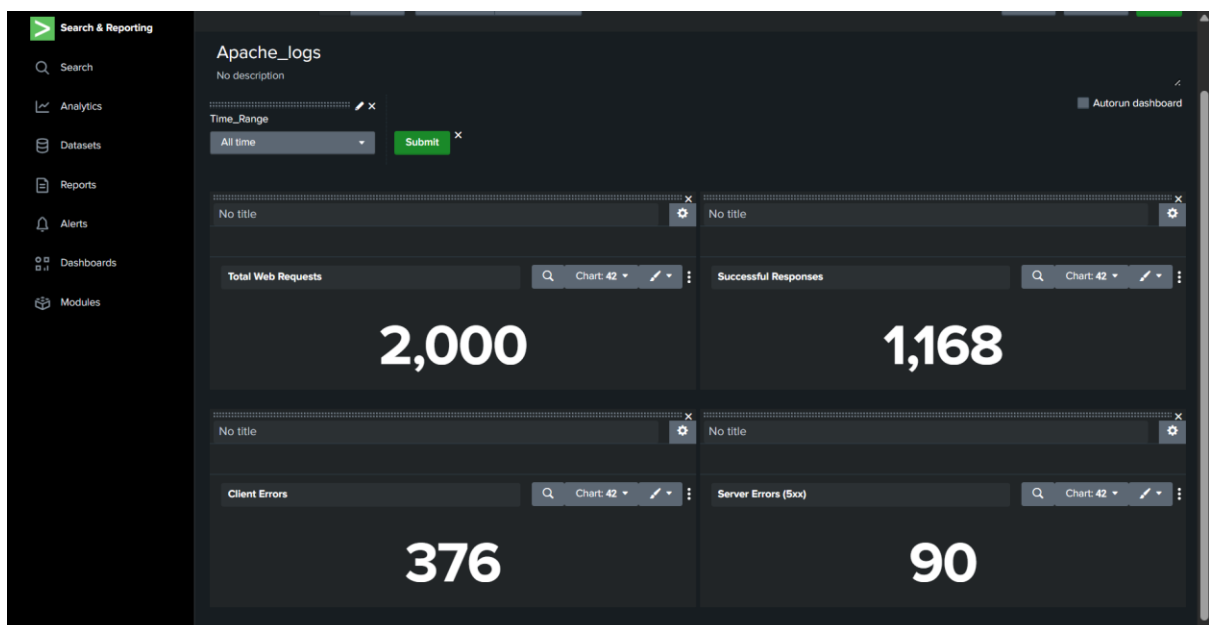
Difference from the Client Error Query:

- The where condition is changed from status>=400 and status<500 to status>=500.
- This filters for **5xx HTTP status codes**, which indicate **server-side failures** rather than client-side issues.
- Examples include:
 - **500** – Internal Server Error
 - **502** – Bad Gateway
 - **503** – Service Unavailable
 - **504** – Gateway Timeout
- The output field is renamed to **"Server Errors"**.



Click on >> Add to Dashboard

We can see Client error and Serve error being displayed



4. Top Requested URIs

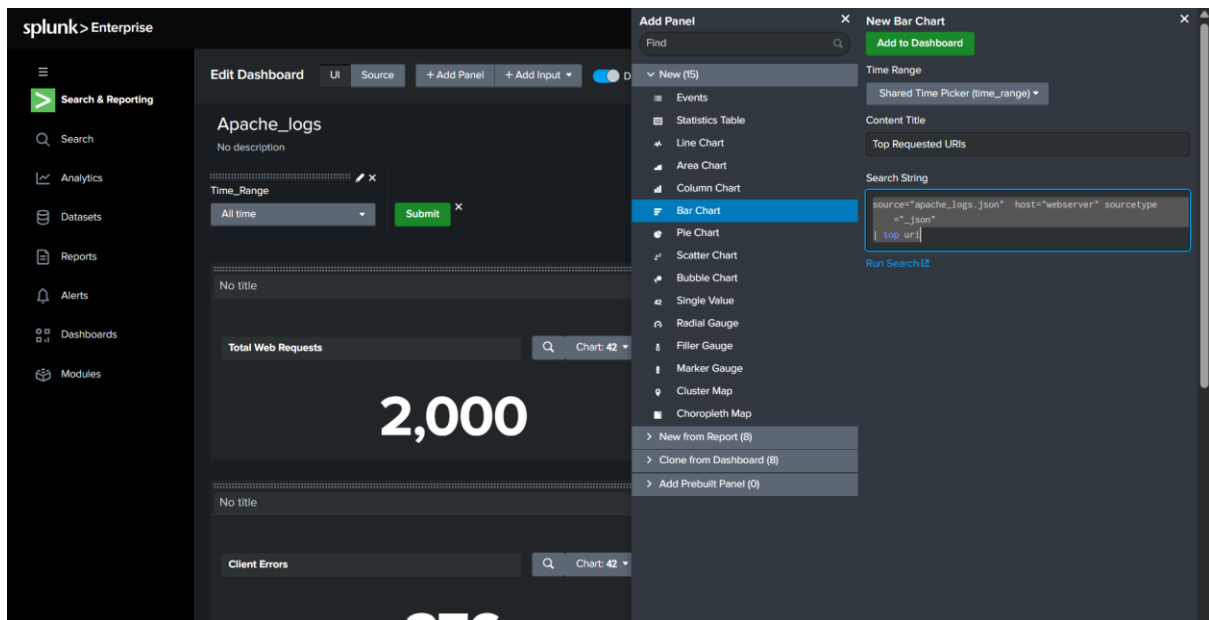
- Click on Add Panel
- Under New, choose Bar Chart
- Use Shared Time Picker **time_range**
- Set Content Title to "Top Requested URIs"

- Enter the Search String as below

```
source="apache_logs.json" host="webserver" sourcetype="_json"
| top uri
```

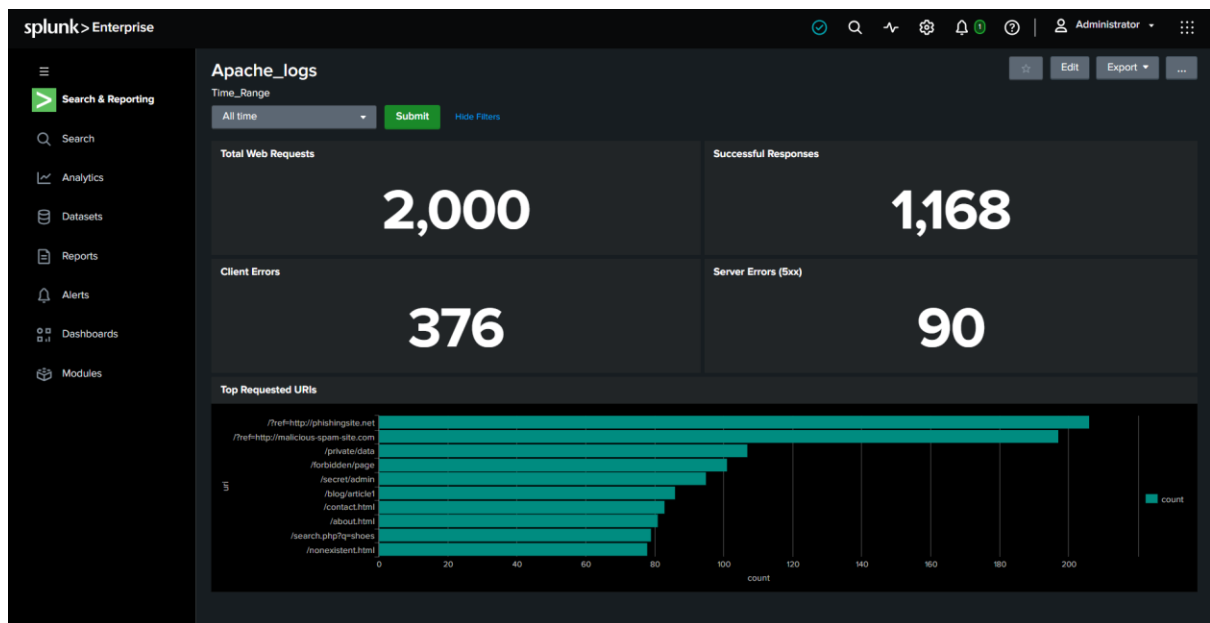
Explanation:

- Filters events from the apache_logs.json log source.
- Limits the search to logs generated by the webserver host.
- Uses the _json sourcetype to process JSON-formatted log data.
- The top command identifies the most frequently occurring values in the uri field.
- The uri field represents the requested resource or endpoint on the web server.
- Splunk automatically calculates the count and percentage of requests for each URI and displays them in descending order.



Click on >> Add to Dashboard

We can see the bar chart in our dashboard



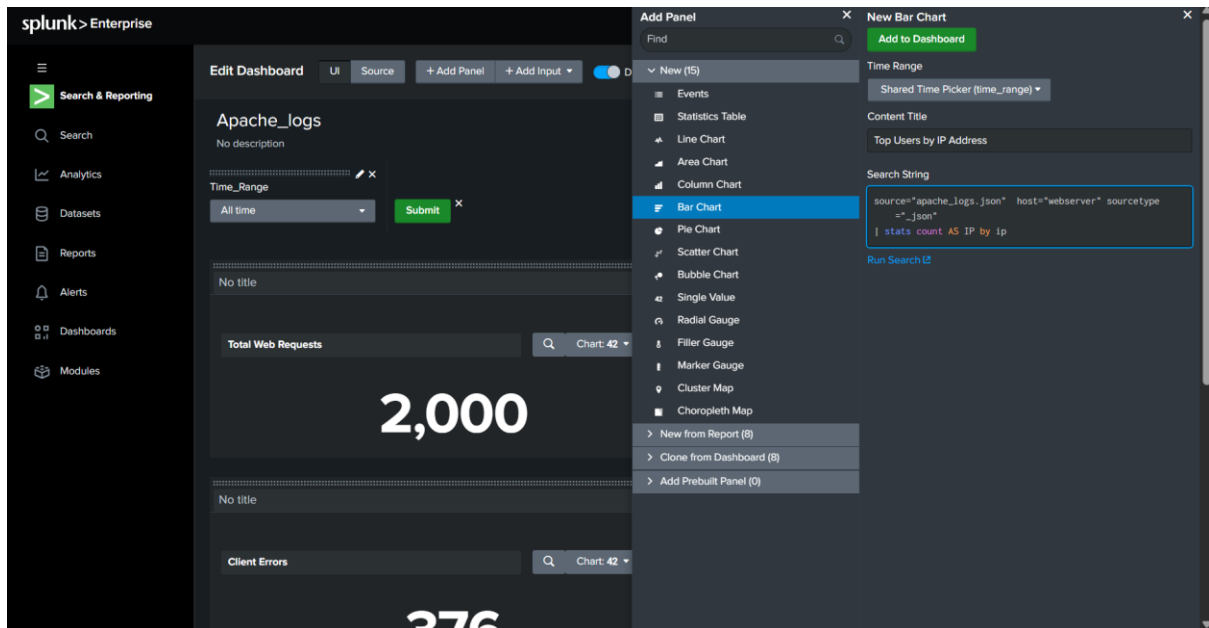
5. Top Users by IP Address

- Click on Add Panel
- Under New, choose Bar Chart
- Use Shared Time Picker **time_range**
- Set Content Title to "Top Users by IP Address"
- Enter the Search String as below

```
source="apache_logs.json" host="webserver" sourcetype="_json"
| stats count AS IP by ip
```

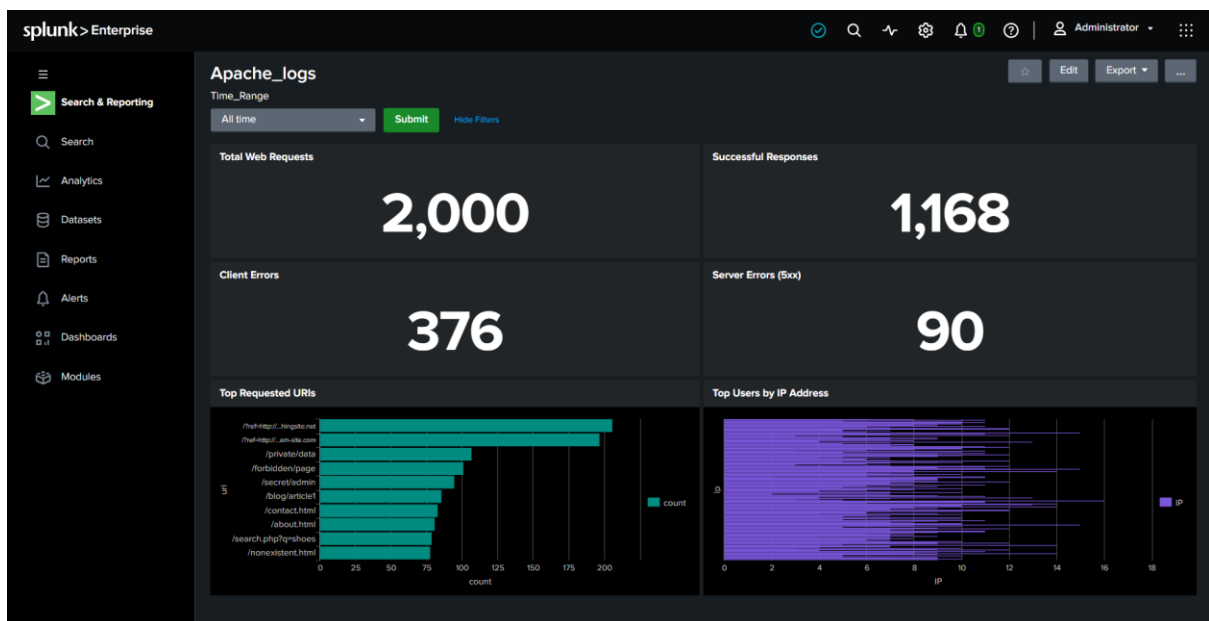
Explanation:

- Filters events from the apache_logs.json log source.
- Limits the search to logs generated by the webserver host.
- Uses the _json sourcetype to process JSON-formatted log data.
- The stats count by ip command groups events based on the client IP address.
- Counts the total number of requests generated by each IP address.
- Renames the count field to **IP** for display purposes.



Click on >> Add to Dashboard

As we can see the chart is visible on our dashboard



6. Web Traffic by Client IP Addresses

- Click on **Add Panel**
- Under **New**, choose **Choropleth Map**
- Use **Shared Time Picker** time_range
- Set **Content Title** to **Web Traffic by Client IP Addresses**
- Enter the **Search String** as below:

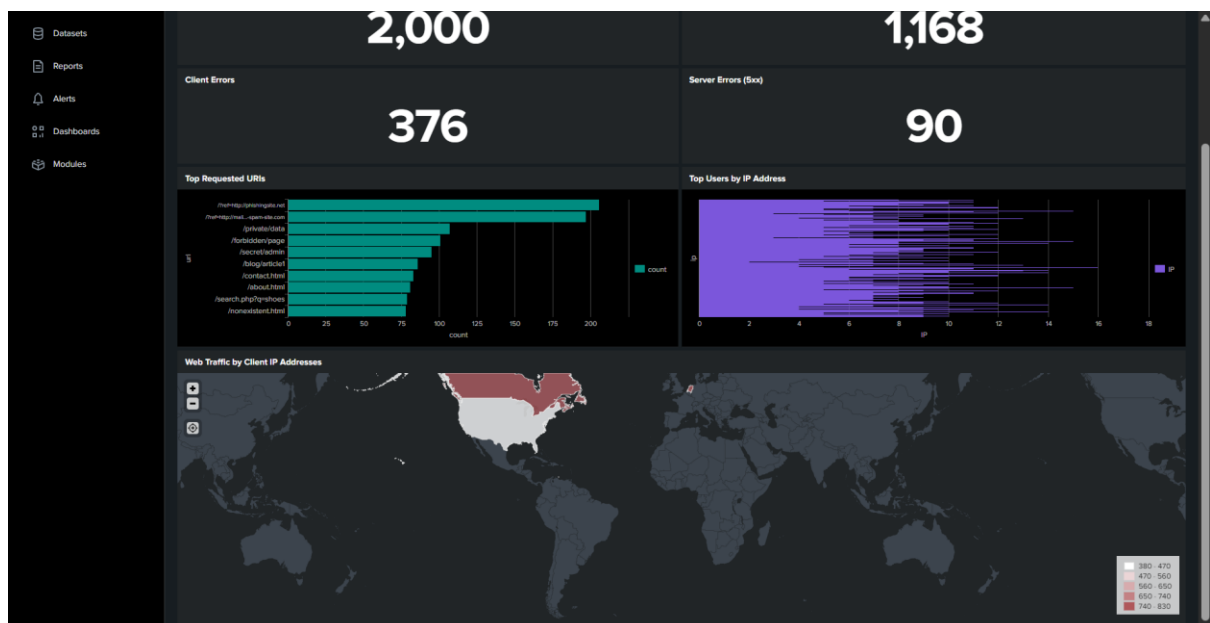

```

source="apache_logs.json" host="webserver" sourcetype="_json"
method=GET
| table ip
| iplocation ip
| stats count by Country
| geom geo_countries featureIdField="Country"

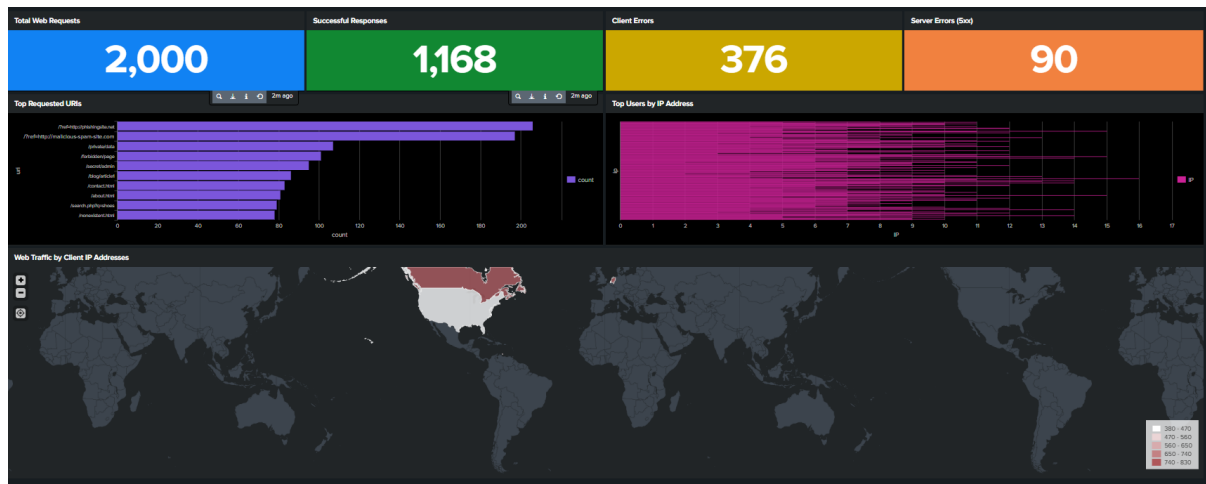
```

Explanation:

- Filters events from the apache_logs.json log source.
- Limits the search to logs generated by the webserver host.
- Uses the _json sourcetype to process JSON-formatted log data.
- Filters for **GET** requests, which represent users requesting web resources.
- The table ip command extracts and displays only the client IP address field.
- The iplocation command performs GeoIP lookups and enriches each IP address with geographic information, including the country.
- The stats count by Country command groups events by country and counts the number of requests originating from each location.
- The geom geo_countries featureIdField="Country" command maps the country data to geographic boundaries, enabling visualization on a choropleth map.



This is the final dashboard



Conclusion

In this lab, I uploaded Apache web server logs into Splunk and built a dashboard with six panels to monitor web activity. The data showed 2,000 total requests, out of which 1,168 were successful GET responses with status 200. However, 376 client-side errors and 90 server-side errors were also recorded, which in a real environment would require immediate investigation — the client errors could point to broken links or unauthorized access attempts, while the server errors may indicate backend instability. The Top Requested URIs panel revealed which pages receive the most traffic, useful for performance optimization. The IP-based panels helped identify which users are most active and where traffic is geographically coming from, with the choropleth map showing the US as a dominant traffic source. This project gave me practical experience in ingesting log data, writing SPL queries, and building visual dashboards that turn raw logs into actionable security and operational insights.